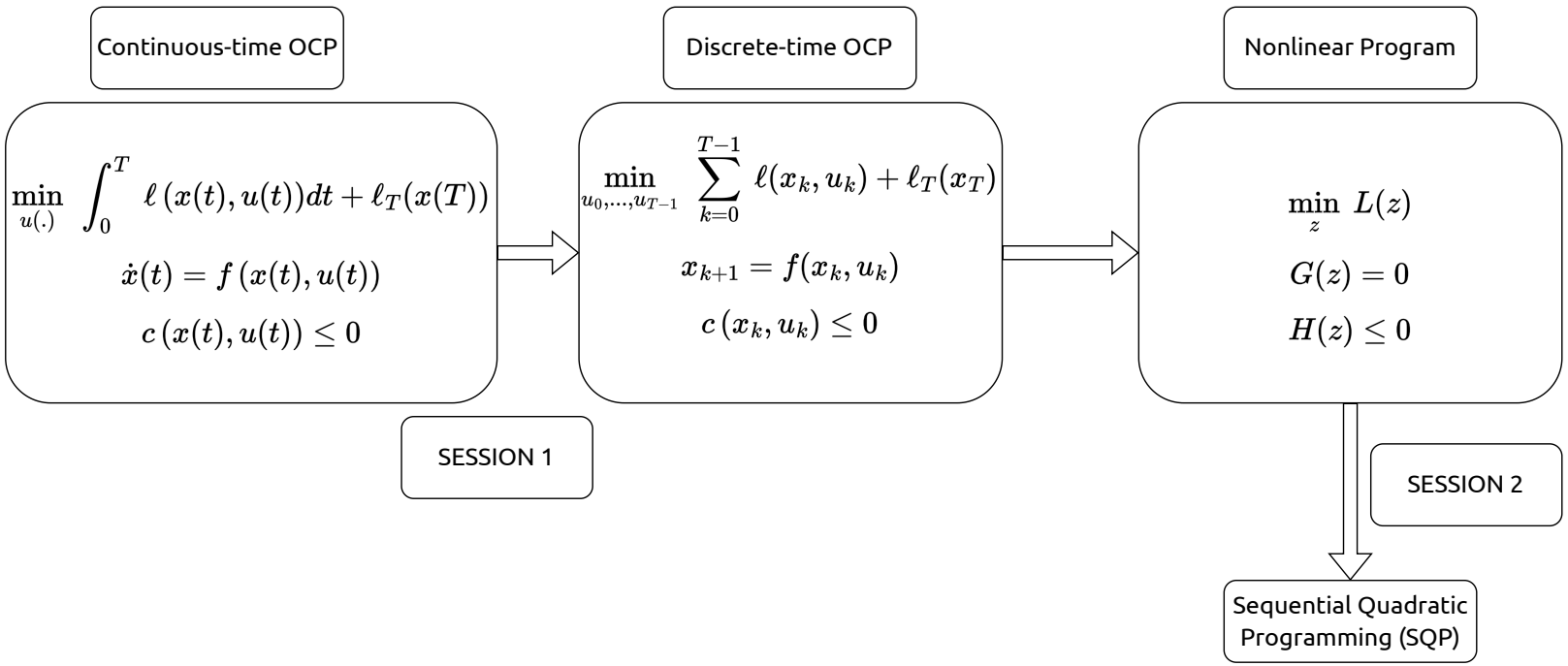
Tutorial on numerical optimal control

Sébastien Kleff

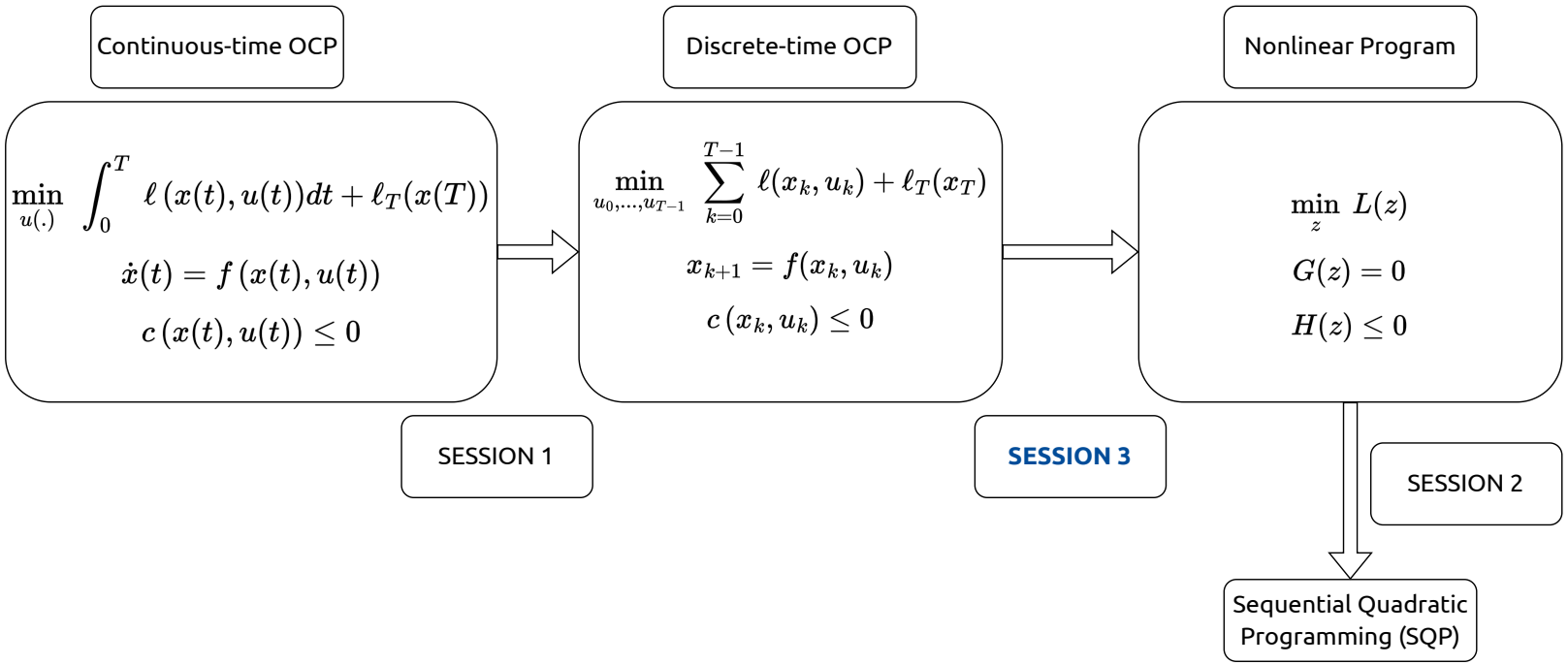
Nov. 14, 2024

SESSION 3

Today


Continuous-time OCP
$\min_{u(.)} \int_0^T \ell\left(x(t), u(t)\right)dt + \ell_T(x(T))$
$\dot{x}(t) = f\left(x(t), u(t)\right)$
$c\left(x(t), u(t)\right) \leq 0$
Discrete-time OCP
$\min_{u_0,\ldots,u_{T-1}} \sum_{k=0}^{T-1} \ell(x_k, u_k) + \ell_T(x_T)$
$x_{k+1} = f(x_k, u_k)$
$c\left(x_k, u_k\right) \leq 0$
Nonlinear Program
$\min_z L(z)$
$G(z) = 0$
$H(z) \leq 0$
SESSION 1
SESSION 2
Sequential Quadratic Programming (SQP)

Today



Today : apply SQP to solve the OCP

- How to go from OCP to NLP ?
- Hands-on tutorial !

Quick reminder on SQP (SESSION 2)

Last time we saw how to solve the **Nonlinear Program** (NLP)

$$\begin{aligned} \min_{z} \; & L(z) \\ \text{s.t. } & G(z) = 0 \\ & H(z) \leq 0 \end{aligned}$$

using the **Sequential Quadratic Programming** (SQP) approach :

Quick reminder on SQP (SESSION 2)

Last time we saw how to solve the **Nonlinear Program** (NLP)

$$\begin{aligned} \min_{z} \; & L(z) \\ \text{s.t. } & G(z) = 0 \\ & H(z) \leq 0 \end{aligned}$$

using the **Sequential Quadratic Programming** (SQP) approach :

1. Quadratize cost, linearize constraint around current iterate (z^i, λ^i, μ^i)

Quick reminder on SQP (SESSION 2)

Last time we saw how to solve the **Nonlinear Program** (NLP)

$$\begin{aligned} \min_{z} \; & L(z) \\ \text{s.t. } & G(z) = 0 \\ & H(z) \leq 0 \end{aligned}$$

using the **Sequential Quadratic Programming** (SQP) approach :

1. Quadratize cost, linearize constraint around current iterate (z^i, λ^i, μ^i)
2. Solve the resulting QP to obtain the step direction $(\Delta z, \Delta\lambda, \Delta\mu)$

$$\left.\begin{aligned} \min_{\Delta z} \; & \frac{1}{2}\Delta z^\top \nabla_{zz}\mathcal{L}(z^i, \lambda^i, \mu^i)\Delta z + \nabla F(z^i)^\top \Delta z \\ \text{s.t. } & \nabla G(z^i)^\top \Delta z + G(z^i) = 0 \\ & \nabla H(z^i)^\top \Delta z + H(z^i) \leq 0 \end{aligned}\right\} \text{ This is a QP}$$

Quick reminder on SQP (SESSION 2)

Last time we saw how to solve the **Nonlinear Program** (NLP)

$$\begin{aligned} \min_{z} \; & L(z) \\ \text{s.t. } & G(z) = 0 \\ & H(z) \leq 0 \end{aligned}$$

using the **Sequential Quadratic Programming** (SQP) approach :

1. Quadratize cost, linearize constraint around current iterate (z^i, λ^i, μ^i)
2. Solve the resulting QP to obtain the step direction $(\Delta z, \Delta\lambda, \Delta\mu)$

$$\left.\begin{aligned} \min_{\Delta z} \; & \frac{1}{2}\Delta z^\top \nabla_{zz}\mathcal{L}(z^i, \lambda^i, \mu^i)\Delta z + \nabla F(z^i)^\top \Delta z \\ \text{s.t. } & \nabla G(z^i)^\top \Delta z + G(z^i) = 0 \\ & \nabla H(z^i)^\top \Delta z + H(z^i) \leq 0 \end{aligned}\right\} \text{ This is a QP}$$

3. Perform a line-search to find the step length α and next iterate

$$(z^{i+1}, \lambda^{i+1}, \mu^{i+1}) = (z^i, \lambda^i, \mu^i) + \alpha(\Delta z, \Delta\lambda, \Delta\mu)$$

From OCP to NLP

Direct optimal control (a.k.a. "first discretize, then optimize") : we want to solve the OCP locally **as an NLP**

$$\min_{u_0,\dots,u_{T-1}} \sum_{k=0}^{T-1} \ell(x_k, u_k) + \ell_T(x_T) \tag{1}$$

$$\text{s.t.} \begin{cases} x_0 = \hat{x} \\ x_{k+1} = f(x_k, u_k) \\ c\left(x_k, u_k\right) \leq 0 \end{cases}$$

There are (at least) 2 ways to transcribe this OCP into an NLP

From OCP to NLP

Direct optimal control (a.k.a. "first discretize, then optimize") : we want to solve the OCP locally **as an NLP**

$$\min_{u_0,\dots,u_{T-1}} \sum_{k=0}^{T-1} \ell(x_k, u_k) + \ell_T(x_T) \tag{1}$$

$$\text{s.t.} \begin{cases} x_0 = \hat{x} \\ x_{k+1} = f(x_k, u_k) \\ c\left(x_k, u_k\right) \leq 0 \end{cases}$$

There are (at least) 2 ways to transcribe this OCP into an NLP

- Direct single shooting ("sequential" approach) : optimize over u only

From OCP to NLP

Direct optimal control (a.k.a. "first discretize, then optimize") : we want to solve the OCP locally **as an NLP**

$$\min_{u_0,\dots,u_{T-1}} \sum_{k=0}^{T-1} \ell(x_k, u_k) + \ell_T(x_T) \tag{1}$$

$$\text{s.t.} \begin{cases} x_0 = \hat{x} \\ x_{k+1} = f(x_k, u_k) \\ c(x_k, u_k) \leq 0 \end{cases}$$

There are (at least) 2 ways to transcribe this OCP into an NLP

- Direct single shooting ("sequential" approach) : optimize over u only
- Direct multiple Shooting ("simultaneous" approach) : optimize over x, u

Direct single shooting

Idea : eliminate the states

Define $z \triangleq (u_0, ..., u_{T-1})$ and simulate ("shoot") the dynamics over $[0, T]$ to re-construct the state trajectory $(x_0, ..., x_T)$ using

$$x_{k+1} = f(x_k, u_k) \qquad \forall k \in \{0, .., T-1\}$$

Direct single shooting

Idea : eliminate the states

Define $z \triangleq (u_0, ..., u_{T-1})$ and simulate ("shoot") the dynamics over $[0, T]$ to re-construct the state trajectory $(x_0, ..., x_T)$ using

$$x_{k+1} = f(x_k, u_k) \qquad \forall k \in \{0, .., T-1\}$$

Advantages

- The dynamics constraint is "naturally" satisfied by the nonlinear rollout
- Reduced problem (KKT matrix is size Tn_u)

Direct single shooting

Idea : eliminate the states

Define $z \triangleq (u_0, ..., u_{T-1})$ and simulate ("shoot") the dynamics over $[0, T]$ to re-construct the state trajectory $(x_0, ..., x_T)$ using

$$x_{k+1} = f(x_k, u_k) \qquad \forall k \in \{0, .., T-1\}$$

Advantages

- The dynamics constraint is "naturally" satisfied by the nonlinear rollout
- Reduced problem (KKT matrix is size Tn_u)

Problem

- Requires a feasible initial guess
- Difficult to add constraints
- Propagates nonlinearities

Direct single shooting

Idea : eliminate the states

Define $z \triangleq (u_0, ..., u_{T-1})$ and simulate ("shoot") the dynamics over $[0, T]$ to re-construct the state trajectory $(x_0, ..., x_T)$ using

$$x_{k+1} = f(x_k, u_k) \qquad \forall k \in \{0, .., T-1\}$$

Advantages

- The dynamics constraint is "naturally" satisfied by the nonlinear rollout
- Reduced problem (KKT matrix is size Tn_u)

Problem

- Requires a feasible initial guess
- Difficult to add constraints
- Propagates nonlinearities

Example : Differential Dynamic Programming (DDP), iLQR

Direct multiple shooting

Idea : keep the states as decision variables

Define $z \triangleq (x_0, u_0, ..., x_{T-1}, u_{T-1}, x_T)$ and simulate the dynamics over each sub-intervals $[t_k, t_{k+1}]$, allowing *gaps*

$$\gamma_{k+1} = f(x_k, u_k) - x_{k+1} \qquad \forall k \in \{0, .., T-1\}$$

Direct multiple shooting

Idea : keep the states as decision variables

Define $z \triangleq (x_0, u_0, ..., x_{T-1}, u_{T-1}, x_T)$ and simulate the dynamics over each sub-intervals $[t_k, t_{k+1}]$, allowing *gaps*

$$\gamma_{k+1} = f(x_k, u_k) - x_{k+1} \qquad \forall k \in \{0, .., T-1\}$$

Advantages

- Can be warm-started with infeasible guess
- Can easily incorporate constraints

Direct multiple shooting

Idea : keep the states as decision variables

Define $z \triangleq (x_0, u_0, ..., x_{T-1}, u_{T-1}, x_T)$ and simulate the dynamics over each sub-intervals $[t_k, t_{k+1}]$, allowing *gaps*

$$\gamma_{k+1} = f(x_k, u_k) - x_{k+1} \qquad \forall k \in \{0, .., T-1\}$$

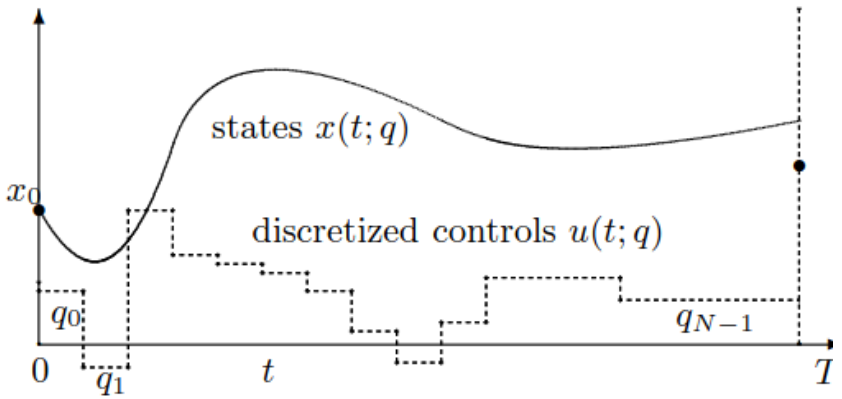
Advantages

- Can be warm-started with infeasible guess
- Can easily incorporate constraints

Example : Feasible DDP (FDDP), Gauss-Newton Multiple Shooting (GNMS)

Single Shooting vs Multiple Shooting

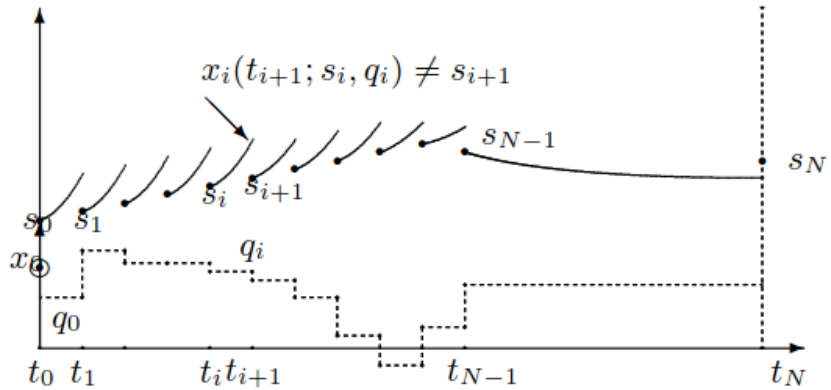
Single shooting



$$\min_{u_0,\ldots,u_{T-1}} \quad (\ldots)$$

Define $z \triangleq (u_0, ..., u_{T-1})$ in the NLP

Multiple shooting



$$\min_{\substack{x_0,\ldots,x_T \\ u_0,\ldots,u_{T-1}}} \quad (\ldots)$$

Define $z \triangleq (x_0, u_0, ..., x_{T-1}, u_{T-1}, x_T)$ in the NLP

Figures from Moritz Diehl, Hans Georg Bock, Holger Diedam, Pierre-Brice Wieber. Fast Direct Multiple Shooting Algorithms for Optimal Robot Control. Fast Motions in Biomechanics and Robotics, 2005, Heidelberg.

Multiple-shooting SQP

Let's apply SQP: the NLP is approximated by a QP around the the current iterate $z^i = (x_0, u_0, ..., x_{T-1}, u_{T-1}, x_T)$

Multiple-shooting SQP

Let's apply SQP: the NLP is approximated by a QP around the the current iterate $z^i = (x_0, u_0, ..., x_{T-1}, u_{T-1}, x_T)$

$$\begin{aligned} \min_{\Delta z} \ & \frac{1}{2}\Delta z^\top \nabla^2 \mathcal{L}(z^i, \lambda^i, \mu^i)\Delta z + \nabla L(z^i)^\top \Delta z \\ \text{s.t.} \ & \nabla G(z^i)^\top \Delta z + G(z^i) = 0 \\ & \nabla H(z^i)^\top \Delta z + H(z^i) \leq 0 \end{aligned} \tag{2}$$

Multiple-shooting SQP

Let's apply SQP: the NLP is approximated by a QP around the the current iterate $z^i = (x_0, u_0, ..., x_{T-1}, u_{T-1}, x_T)$

$$\begin{aligned} \min_{\Delta z} \ & \frac{1}{2}\Delta z^\top \nabla^2 \mathcal{L}(z^i, \lambda^i, \mu^i)\Delta z + \nabla L(z^i)^\top \Delta z \\ \text{s.t. } & \nabla G(z^i)^\top \Delta z + G(z^i) = 0 \\ & \nabla H(z^i)^\top \Delta z + H(z^i) \leq 0 \end{aligned} \tag{2}$$

The key is simply to define $\mathcal{L}, L, G, H$ appropriately in terms of ℓ_k, f and c.

Multiple-shooting SQP

Let's apply SQP: the NLP is approximated by a QP around the the current iterate $z^i = (x_0, u_0, ..., x_{T-1}, u_{T-1}, x_T)$

$$\begin{aligned} \min_{\Delta z} \ & \frac{1}{2}\Delta z^\top \nabla^2 \mathcal{L}(z^i, \lambda^i, \mu^i)\Delta z + \nabla L(z^i)^\top \Delta z \\ \text{s.t. } & \nabla G(z^i)^\top \Delta z + G(z^i) = 0 \\ & \nabla H(z^i)^\top \Delta z + H(z^i) \leq 0 \end{aligned} \tag{2}$$

The key is simply to define $\mathcal{L}, L, G, H$ appropriately in terms of ℓ_k, f and c.

Easy to do, especially because the Lagrangian is separable

$$\mathcal{L}(z^i, \lambda^i, \mu^i) = \sum_{k=0}^{T-1} \underbrace{\mathcal{L}_k(z_k^i, \lambda_k^i, \mu_k^i)}_{\text{Stagewise Lagrangian}} + \underbrace{\mathcal{L}_T(x_T^i, \mu_T^i)}_{\text{Terminal Lagrangian}} \tag{3}$$

Multiple-shooting SQP

Let's apply SQP: the NLP is approximated by a QP around the the current iterate $z^i = (x_0, u_0, ..., x_{T-1}, u_{T-1}, x_T)$

$$\begin{aligned} \min_{\Delta z} \ & \frac{1}{2}\Delta z^\top \nabla^2 \mathcal{L}(z^i, \lambda^i, \mu^i)\Delta z + \nabla L(z^i)^\top \Delta z \\ \text{s.t. } & \nabla G(z^i)^\top \Delta z + G(z^i) = 0 \\ & \nabla H(z^i)^\top \Delta z + H(z^i) \leq 0 \end{aligned} \tag{2}$$

The key is simply to define $\mathcal{L}, L, G, H$ appropriately in terms of ℓ_k, f and c.

Easy to do, especially because the Lagrangian is separable

$$\mathcal{L}(z^i, \lambda^i, \mu^i) = \sum_{k=0}^{T-1} \underbrace{\mathcal{L}_k(z_k^i, \lambda_k^i, \mu_k^i)}_{\text{Stagewise Lagrangian}} + \underbrace{\mathcal{L}_T(x_T^i, \mu_T^i)}_{\text{Terminal Lagrangian}} \tag{3}$$

$\nabla L, \nabla G, \nabla H$ are expressed in terms of $\nabla \ell_k, \nabla f, \nabla c$,

$\nabla^2 \mathcal{L}$ involves second-order derivatives $\nabla^2 \ell_k, \nabla^2 f, \nabla^2 c$

Multiple-shooting SQP

Writing out the SQP is not hard, but it's a little bit tedious...

$$\min_{\Delta \boldsymbol{x}, \Delta \boldsymbol{u}} \sum_{k=0}^{T-1} \frac{1}{2} \begin{bmatrix} \Delta x_k \\ \Delta u_k \end{bmatrix}^\top \begin{bmatrix} Q_k & S_k \\ S_k^\top & R_k \end{bmatrix} \begin{bmatrix} \Delta x_k \\ \Delta u_k \end{bmatrix} + \begin{bmatrix} q_k \\ r_k \end{bmatrix}^\top \begin{bmatrix} \Delta x_k \\ \Delta u_k \end{bmatrix} + \frac{1}{2} \Delta x_T^\top Q_T \Delta x_T + \Delta x_T^\top q_T \tag{4a}$$

$$\text{s.t.} \Delta x_{k+1} = A_k \Delta x_k + B_k \Delta u_k + \gamma_{k+1}, 0 \leq k < T, \tag{4b}$$

$$D_k \Delta x_k + E_k \Delta u_k + c_k \geq 0, \qquad 0 \leq k < T. \tag{4c}$$

$$D_T \Delta x_T + c_T \geq 0. \tag{4d}$$

where $q_T = \nabla_x \ell_T(x_T^{[n]})$ and

$$\begin{aligned} Q_T &= \left(\nabla_{xx}^2 \ell_T - \mu_T^\top \nabla_{xx}^2 c_T \right) (x_T^{[n]}) \\ Q_k &= \left(\nabla_{xx}^2 \ell_k + \lambda_{k+1}^\top \nabla_{xx}^2 f_k - \mu_k^\top \nabla_{xx}^2 c_k \right) (x_k^{[n]}, u_k^{[n]}) \\ S_k &= \left(\nabla_{xu}^2 \ell_k + \lambda_{k+1}^\top \nabla_{xu}^2 f_k - \mu_k^\top \nabla_{xu}^2 c_k \right) (x_k^{[n]}, u_k^{[n]}) \\ R_k &= \left(\nabla_{uu}^2 \ell_k + \lambda_{k+1}^\top \nabla_{uu}^2 f_k - \mu_k^\top \nabla_{uu}^2 c_k \right) (x_k^{[n]}, u_k^{[n]}) \end{aligned} \tag{5}$$

$$\begin{aligned} A_k &= \nabla_x f_k(x_k^{[n]}, u_k^{[n]}), \quad B_k = \nabla_u f_k(x_k^{[n]}, u_k^{[n]}) \quad 0 \leq k < T. \\ q_k &= \nabla_x \ell_k(x_k^{[n]}, u_k^{[n]}), \quad r_k = \nabla_u \ell_k(x_k^{[n]}, u_k^{[n]}) \\ D_k &= \nabla_x c_k(x_k^{[n]}, u_k^{[n]}), \quad E_k = \nabla_u c_k(x_k^{[n]}, u_k^{[n]}) \\ c_k &= c_k(x_k^{[n]}, u_k^{[n]}), \qquad D_T = \nabla_x c_T(x_k^{[n]}) \end{aligned} \tag{6}$$

Hand-on: Solve your first OCP

You need to have a working conda environment beyond this point !

We will use the `Crocoddyl` library to formulate and transcript the OCP

- Define the dynamics model, cost function & constraints
- Define the integration method

Hand-on: Solve your first OCP

You need to have a working conda environment beyond this point !

We will use the `Crocoddyl` library to formulate and transcript the OCP

- Define the dynamics model, cost function & constraints
- Define the integration method

The pendulum swing-up task :

State $x = (\theta, \dot{\theta})$, control $u = \tau$ and dynamics model f

$$ml^2\ddot{\theta} + mgl\sin\theta = \tau \tag{7}$$

Running cost $\ell(x, u) = \alpha\|u\|^2$ and terminal cost $\ell_T(x) = \|x\|^2$

Hand-on: Solve your first OCP

You need to have a working conda environment beyond this point !

We will use the `Crocoddyl` library to formulate and transcript the OCP

- Define the dynamics model, cost function & constraints
- Define the integration method

The pendulum swing-up task :

State $x = (\theta, \dot{\theta})$, control $u = \tau$ and dynamics model f

$$ml^2\ddot{\theta} + mgl\sin\theta = \tau \tag{7}$$

Running cost $\ell(x, u) = \alpha\|u\|^2$ and terminal cost $\ell_T(x) = \|x\|^2$

Your turn ! Write down the cost, dynamics and derivatives

`pendulum/pendulum_ocp.py`

Hand-on: Cartpole

Adapted from the cartpole notebook in `Crocoddyl` Your turn ! Write down the cost, dynamics and derivatives

`cartpole/cartpole_ocp.py`

Example: Kuka

Let's apply what we've seen so far to the Kuka robot

Example: Kuka

Let's apply what we've seen so far to the Kuka robot

- **State** : joint position-velocity $x = (q, v) \in \mathbb{R}^{14}$

Example: Kuka

Let's apply what we've seen so far to the Kuka robot

- **State** : joint position-velocity $x = (q, v) \in \mathbb{R}^{14}$
- **Control** : joint torques $\tau \in \mathbb{R}^7$

Example: Kuka

Let's apply what we've seen so far to the Kuka robot

- **State** : joint position-velocity $x = (q, v) \in \mathbb{R}^{14}$
- **Control** : joint torques $\tau \in \mathbb{R}^7$
- **Dynamics** :

$$M(q)\ddot{q} + h(q, \dot{q}) = \tau$$

Example: Kuka

Let's apply what we've seen so far to the Kuka robot

- **State** : joint position-velocity $x = (q, v) \in \mathbb{R}^{14}$
- **Control** : joint torques $\tau \in \mathbb{R}^7$
- **Dynamics** :

$$M(q)\ddot{q} + h(q, \dot{q}) = \tau$$

- **Cost function** : encoding a reaching task

$$\ell(x, u) = w_1 \|p(q) - \bar{p}\|^2 + w_2 \|x - \bar{x}\|^2 + w_3 \|u\|^2$$

where $p(q)$ is the forward kinematics map and $w_1, w_2, w_3 > 0$

Example: Kuka

Let's apply what we've seen so far to the Kuka robot

- **State** : joint position-velocity $x = (q, v) \in \mathbb{R}^{14}$
- **Control** : joint torques $\tau \in \mathbb{R}^7$
- **Dynamics** :

$$M(q)\ddot{q} + h(q, \dot{q}) = \tau$$

- **Cost function** : encoding a reaching task

$$\ell(x, u) = w_1 \|p(q) - \bar{p}\|^2 + w_2 \|x - \bar{x}\|^2 + w_3 \|u\|^2$$

  where $p(q)$ is the forward kinematics map and $w_1, w_2, w_3 > 0$
- We use semi-implicit Euler integration scheme with $\mathrm{d}t = 20\text{ms}$
- $T = 50$ nodes in the OCP horizon

Example: Kuka

Let's apply what we've seen so far to the Kuka robot

- **State** : joint position-velocity $x = (q, v) \in \mathbb{R}^{14}$
- **Control** : joint torques $\tau \in \mathbb{R}^7$
- **Dynamics** :

$$M(q)\ddot{q} + h(q, \dot{q}) = \tau$$

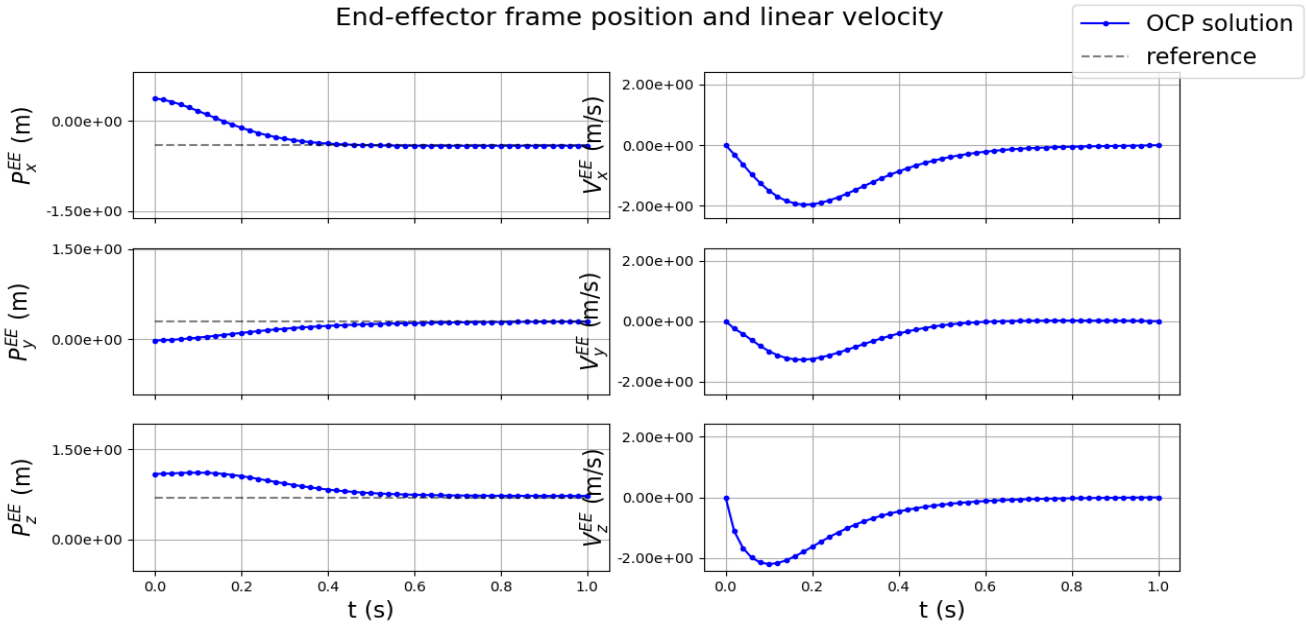
- **Cost function** : encoding a reaching task

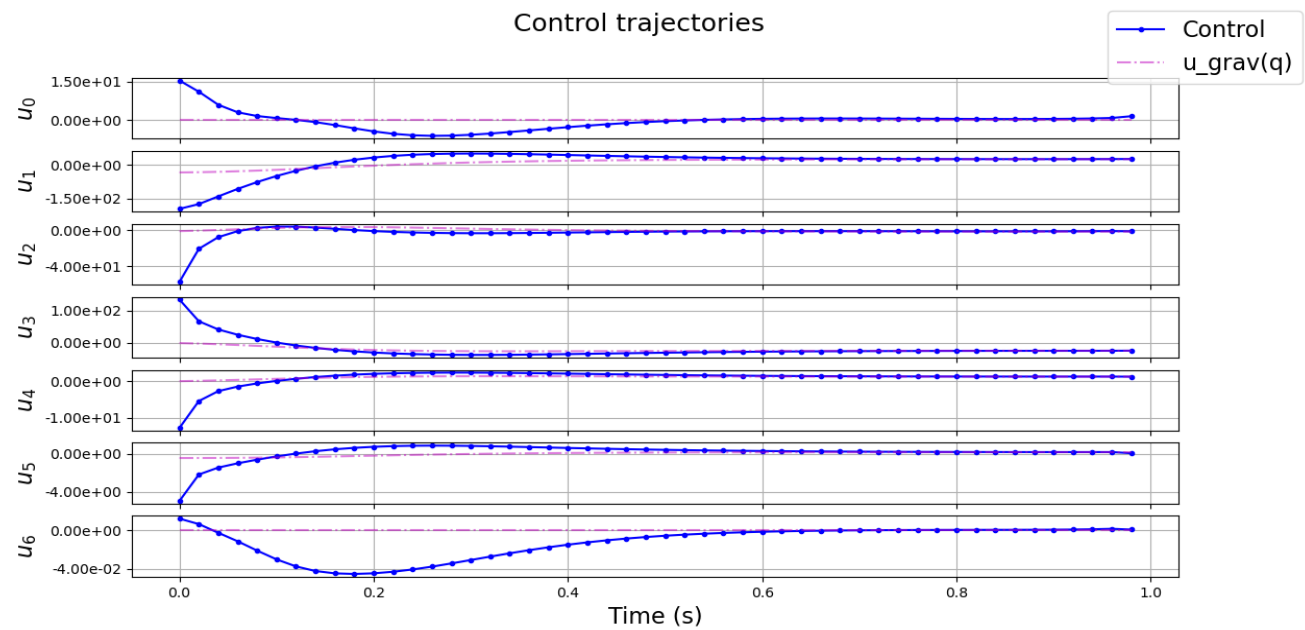
$$\ell(x, u) = w_1 \|p(q) - \bar{p}\|^2 + w_2 \|x - \bar{x}\|^2 + w_3 \|u\|^2$$

  where $p(q)$ is the forward kinematics map and $w_1, w_2, w_3 > 0$
- We use semi-implicit Euler integration scheme with $\mathrm{d}t = 20\text{ms}$
- $T = 50$ nodes in the OCP horizon

https://github.com/machines-in-motion/minimal_examples_crocoddyl/blob/master/ocp_kuka_reaching.py

Example: Kuka





Example: Kuka

The solution looks nice

Example: Kuka

The solution looks nice

Let's run the optimal control sequence $u_0, ..., u_{T-1}$ in a simulator

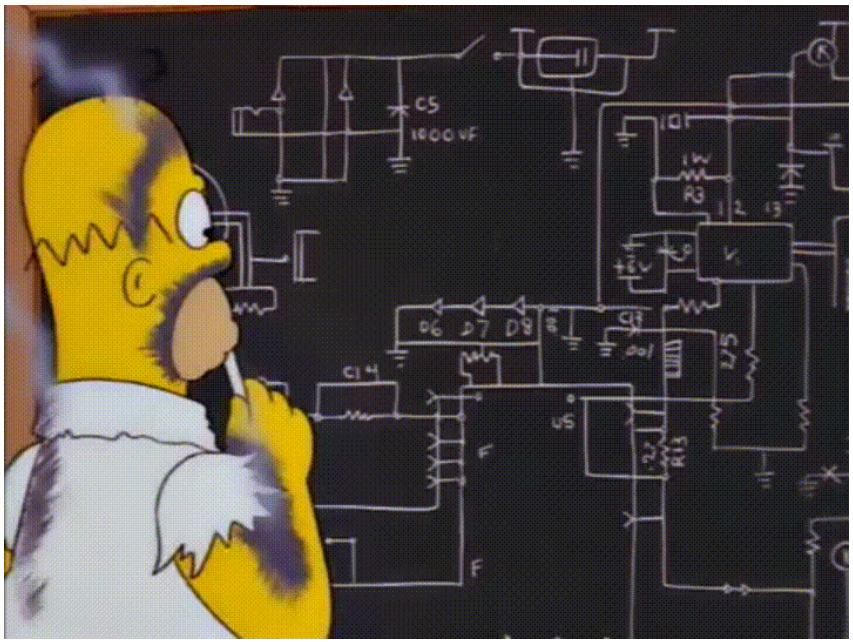
https://github.com/machines-in-motion/minimal_examples_crocoddyl/blob/master/mpc_kuka_reaching.py

Example: Kuka

The solution looks nice

Let's run the optimal control sequence $u_0, ..., u_{T-1}$ in a simulator

https://github.com/machines-in-motion/minimal_examples_crocoddyl/blob/master/mpc_kuka_reaching.py

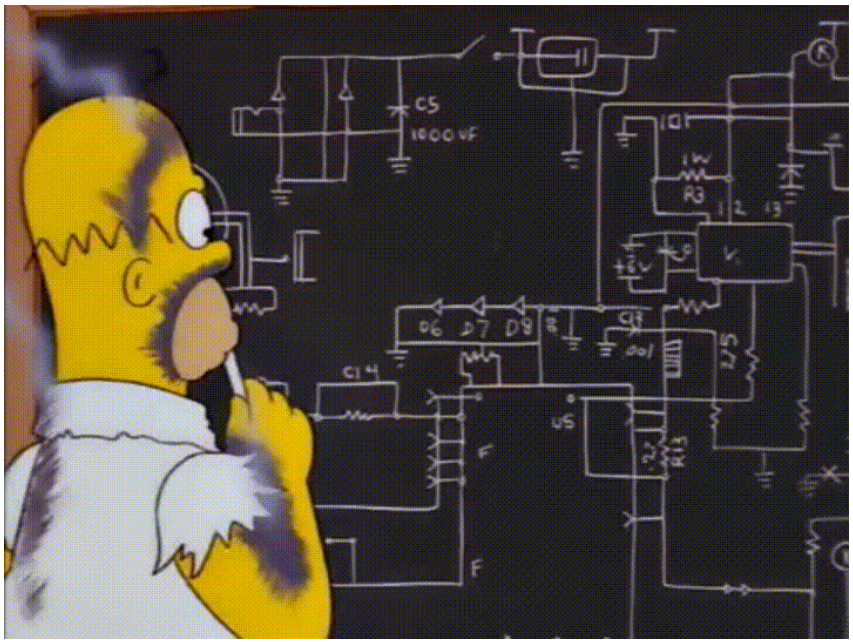


Example: Kuka

The solution looks nice

Let's run the optimal control sequence $u_0, ..., u_{T-1}$ in a simulator

https://github.com/machines-in-motion/minimal_examples_crocoddyl/blob/master/mpc_kuka_reaching.py



We need to re-plan !

Model-Predictive Control (MPC)

Idea : This is a state-feedback controller constructed by repeatedly computing open-loop control sequences $u(t)$ online

Model-Predictive Control (MPC)

Idea : This is a state-feedback controller constructed by repeatedly computing open-loop control sequences $u(t)$ online

1. Collect current state estimate $\hat{x}$

Model-Predictive Control (MPC)

Idea : This is a state-feedback controller constructed by repeatedly computing open-loop control sequences $u(t)$ online

1. Collect current state estimate $\hat{x}$
2. Solve NLP with initial conition $x_0 = \hat{x}$

Model-Predictive Control (MPC)

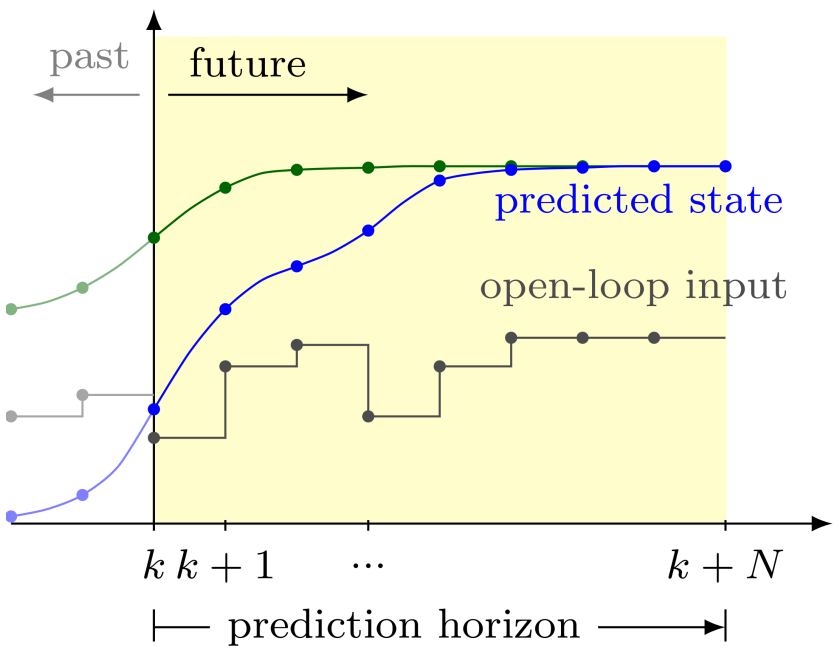
Idea : This is a state-feedback controller constructed by repeatedly computing open-loop control sequences $u(t)$ online

1. Collect current state estimate $\hat{x}$
2. Solve NLP with initial conition $x_0 = \hat{x}$
3. Apply the first optimal control u_0

Model-Predictive Control (MPC)

Idea : This is a state-feedback controller constructed by repeatedly computing open-loop control sequences $u(t)$ online

1. Collect current state estimate $\hat{x}$
2. Solve NLP with initial conition $x_0 = \hat{x}$
3. Apply the first optimal control u_0



Model-Predictive Control (MPC)

A few remarks

- MPC = approximation of the closed-loop optimal policy $\pi(x)$

Model-Predictive Control (MPC)

A few remarks

- MPC = approximation of the closed-loop optimal policy $\pi(x)$
- MPC is rich research subject on its own! Read Mayne, Grüne, etc.

Model-Predictive Control (MPC)

A few remarks

- MPC = approximation of the closed-loop optimal policy $\pi(x)$
- MPC is rich research subject on its own! Read Mayne, Grüne, etc.
- Very interesting for robotics: automatize complex motion synthesis, enforce constraints directly at the control design level

Model-Predictive Control (MPC)

A few remarks

- MPC = approximation of the closed-loop optimal policy $\pi(x)$
- MPC is rich research subject on its own! Read Mayne, Grüne, etc.
- Very interesting for robotics: automatize complex motion synthesis, enforce constraints directly at the control design level

Problem : online computation $\Longrightarrow$ need a solution quickly

A little bit of complexity

The MPC solving time depends on the NLP size, i.e.

- Horizon length T
- State and control dimensions n_x, n_u

A little bit of complexity

The MPC solving time depends on the NLP size, i.e.

- Horizon length T
- State and control dimensions n_x, n_u

In our case (Kuka) : $n_x = 14, n_u = 7, T = 50$ so $z \in \mathbb{R}^{1064}$!

A little bit of complexity

The MPC solving time depends on the NLP size, i.e.

- Horizon length T
- State and control dimensions n_x, n_u

In our case (Kuka) : $n_x = 14, n_u = 7, T = 50$ so $z \in \mathbb{R}^{1064}$!

We *need* to design efficient solvers

A little bit of complexity

The MPC solving time depends on the NLP size, i.e.

- Horizon length T
- State and control dimensions n_x, n_u

In our case (Kuka) : $n_x = 14, n_u = 7, T = 50$ so $z \in \mathbb{R}^{1064}$!

We *need* to design efficient solvers

Nice fact : The OCP induces a particular structure in the NLP that can be exploited in the resolution.

Exploiting sparsity

Remember this ?

Exploiting sparsity

Remember this ?

$$\min_{\Delta \boldsymbol{x}, \Delta \boldsymbol{u}} \sum_{k=0}^{T-1} \frac{1}{2} \begin{bmatrix} \Delta x_k \\ \Delta u_k \end{bmatrix}^\top \begin{bmatrix} Q_k & S_k \\ S_k^\top & R_k \end{bmatrix} \begin{bmatrix} \Delta x_k \\ \Delta u_k \end{bmatrix} + \begin{bmatrix} q_k \\ r_k \end{bmatrix}^\top \begin{bmatrix} \Delta x_k \\ \Delta u_k \end{bmatrix} + \frac{1}{2} \Delta x_T^\top Q_T \Delta x_T + \Delta x_T^\top q_T \tag{8a}$$

$$\text{s.t.} \Delta x_{k+1} = A_k \Delta x_k + B_k \Delta u_k + \gamma_{k+1}, 0 \leq k < T, \tag{8b}$$

$$D_k \Delta x_k + E_k \Delta u_k + c_k \geq 0, \qquad 0 \leq k < T. \tag{8c}$$

$$D_T \Delta x_T + c_T \geq 0. \tag{8d}$$

where $q_T = \nabla_x \ell_T(x_T^{[n]})$ and

$$Q_T = \left(\nabla^2_{xx} \ell_T - \mu_T^\top \nabla^2_{xx} c_T \right) (x_T^{[n]}) \tag{9}$$

$$Q_k = \left(\nabla^2_{xx} \ell_k + \lambda_{k+1}^\top \nabla^2_{xx} f_k - \mu_k^\top \nabla^2_{xx} c_k \right) (x_k^{[n]}, u_k^{[n]})$$

$$S_k = \left(\nabla^2_{xu} \ell_k + \lambda_{k+1}^\top \nabla^2_{xu} f_k - \mu_k^\top \nabla^2_{xu} c_k \right) (x_k^{[n]}, u_k^{[n]})$$

$$R_k = \left(\nabla^2_{uu} \ell_k + \lambda_{k+1}^\top \nabla^2_{uu} f_k - \mu_k^\top \nabla^2_{uu} c_k \right) (x_k^{[n]}, u_k^{[n]})$$

$$A_k = \nabla_x f_k(x_k^{[n]}, u_k^{[n]}), \quad B_k = \nabla_u f_k(x_k^{[n]}, u_k^{[n]}) \quad 0 \leq k < T.$$

$$q_k = \nabla_x \ell_k(x_k^{[n]}, u_k^{[n]}), \quad r_k = \nabla_u \ell_k(x_k^{[n]}, u_k^{[n]})$$

$$D_k = \nabla_x c_k(x_k^{[n]}, u_k^{[n]}), \quad E_k = \nabla_u c_k(x_k^{[n]}, u_k^{[n]})$$

$$c_k = c_k(x_k^{[n]}, u_k^{[n]}), \qquad D_T = \nabla_x c_T(x_k^{[n]}) \tag{10}$$

Exploiting sparsity

Let's write the KKT conditions of this QP

Exploiting sparsity

Let's write the KKT conditions of this QP

$$\begin{bmatrix} \Gamma_1 & M_1^\top & 0 & 0 & \cdots & 0 \\ M_1 & \Gamma_2 & M_2^\top & 0 & \cdots & 0 \\ 0 & M_2 & \Gamma_3 & M_3^\top & \cdots & 0 \\ 0 & 0 & M_3 & \Gamma_4 & \ddots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \ddots & \Gamma_T \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ \vdots \\ s_T \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \\ \vdots \\ g_T \end{bmatrix}$$

Exploiting sparsity

Let's write the KKT conditions of this QP

$$\begin{bmatrix} \Gamma_1 & M_1^\top & 0 & 0 & \cdots & 0 \\ M_1 & \Gamma_2 & M_2^\top & 0 & \cdots & 0 \\ 0 & M_2 & \Gamma_3 & M_3^\top & \cdots & 0 \\ 0 & 0 & M_3 & \Gamma_4 & \ddots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \ddots & \Gamma_T \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ \vdots \\ s_T \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \\ \vdots \\ g_T \end{bmatrix}$$

Naive inversion is $O(T^3)$

Exploiting sparsity

Let's write the KKT conditions of this QP

$$\begin{bmatrix} \Gamma_1 & M_1^\top & 0 & 0 & \cdots & 0 \\ M_1 & \Gamma_2 & M_2^\top & 0 & \cdots & 0 \\ 0 & M_2 & \Gamma_3 & M_3^\top & \cdots & 0 \\ 0 & 0 & M_3 & \Gamma_4 & \ddots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \ddots & \Gamma_T \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ \vdots \\ s_T \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \\ \vdots \\ g_T \end{bmatrix}$$

Naive inversion is $O(T^3)$

The KKT matrix of our QP has a **tri-diagonal structure**

Exploiting sparsity

Let's write the KKT conditions of this QP

$$\begin{bmatrix} \Gamma_1 & M_1^\top & 0 & 0 & \cdots & 0 \\ M_1 & \Gamma_2 & M_2^\top & 0 & \cdots & 0 \\ 0 & M_2 & \Gamma_3 & M_3^\top & \cdots & 0 \\ 0 & 0 & M_3 & \Gamma_4 & \ddots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \ddots & \Gamma_T \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ \vdots \\ s_T \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \\ \vdots \\ g_T \end{bmatrix}$$

Naive inversion is $O(T^3)$

The KKT matrix of our QP has a **tri-diagonal structure**

Can be inverted *efficiently* by using LQR with $O(T)$ complexity !

Exploiting sparsity

Let's write the KKT conditions of this QP

$$\begin{bmatrix} \Gamma_1 & M_1^\top & 0 & 0 & \cdots & 0 \\ M_1 & \Gamma_2 & M_2^\top & 0 & \cdots & 0 \\ 0 & M_2 & \Gamma_3 & M_3^\top & \cdots & 0 \\ 0 & 0 & M_3 & \Gamma_4 & \ddots & 0 \\ \vdots & \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \ddots & \Gamma_T \end{bmatrix} \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ \vdots \\ s_T \end{bmatrix} = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \\ g_4 \\ \vdots \\ g_T \end{bmatrix}$$

Naive inversion is $O(T^3)$

The KKT matrix of our QP has a **tri-diagonal structure**

Can be inverted *efficiently* by using LQR with $O(T)$ complexity !

Riccati recursions of LQR = efficient QP resolution

Efficient SQP for MPC

Tailored efficient SQP for MPC

1. Collect the state estimate $\hat{x}$
2. Compute LQ approximation of the NLP around the current guess z
3. Solve the QP *efficiently* to get the step direction $\Delta z = (\Delta x, \Delta u)$
4. Line-search & update iterate
5. Apply first optimal control u_0